

UNIVERSIDAD SAN SEBASTIÁN

Facultad de Ingeniería, Arquitectura y Diseño

SOLEMNE II

Proyecto Final — DataViz Python Lab

Construyendo Interfaces de Datos Interactivas con APIs REST

Asignatura:	FITO9017 — Programación en Python
Unidad / Semana:	Unidad 3 — Semana 13
Tipo de trabajo:	Grupal (3 personas)
Ponderación:	40 % de la asignatura
Integrantes:	[Completar nombres del equipo]
Docente:	[Nombre del docente]
Fecha de entrega:	Semana 13

Índice

1.	Resumen ejecutivo	3
2.	Descripción del desafío	3
3.	Fuente de datos seleccionada	4
4.	Arquitectura de la solución	5
5.	Proceso desarrollado	6
6.	Código Python (descripción)	7
7.	Aplicación web Streamlit	9
8.	Análisis de datos y hallazgos	10
9.	Pruebas de usabilidad	11
10.	Aprendizajes obtenidos	12
11.	Instrucciones de ejecución	12
12.	Conclusiones	13
13.	Referencias	13

1. Resumen ejecutivo

Este informe documenta el desarrollo del Proyecto Final de la asignatura **FITO9017 — Programación en Python**, correspondiente a la evaluación Solemne II. El proyecto consistió en construir una aplicación web interactiva, denominada **DataViz Chile**, capaz de consumir la API REST pública del Portal de Datos Abiertos del Gobierno de Chile (<https://datos.gob.cl>), basada en CKAN, para permitir a cualquier usuario buscar, explorar, analizar y visualizar datasets públicos de manera intuitiva.

La solución integra las cinco librerías exigidas por el enunciado (**requests**, **json**, **pandas**, **matplotlib** y **streamlit**) y entrega una interfaz multi-página, con filtros dinámicos, métricas automáticas, gráficos interactivos (barras, líneas, torta e histogramas) y descarga de resultados en CSV.

2. Descripción del desafío

El enunciado del Solemne II plantea desarrollar un programa en Python que interactúe con una API REST pública para seleccionar, analizar y presentar datos provenientes del portal del Gobierno de Chile. El trabajo debe culminar en una aplicación web construida con Streamlit que permita la interacción del usuario con los datos (filtros, selección, navegación entre vistas).

Objetivos del proyecto

- **Integración y aplicación de conocimientos:** aplicar los contenidos de las Unidades 1, 2 y 3 (sintaxis, funciones, estructuras de datos, POO, manejo de excepciones, archivos) en un contexto práctico real.
- **Interacción con APIs REST:** demostrar dominio en la utilización del protocolo HTTP para obtener datos de un servicio web.
- **Análisis de datos:** procesar, transformar y resumir información tabular con la librería pandas.
- **Presentación de resultados:** entregar una aplicación web visual e interactiva que comunique los hallazgos de forma comprensible.

3. Fuente de datos seleccionada

Se seleccionó como fuente el **Portal de Datos Abiertos del Gobierno de Chile** (<https://datos.gob.cl>), que utiliza la plataforma **CKAN** y expone una API REST documentada bajo la ruta `/api/3/action/`. Este portal agrupa datasets de múltiples instituciones públicas chilenas en categorías como clima, salud, educación, economía, transporte, justicia, entre otras.

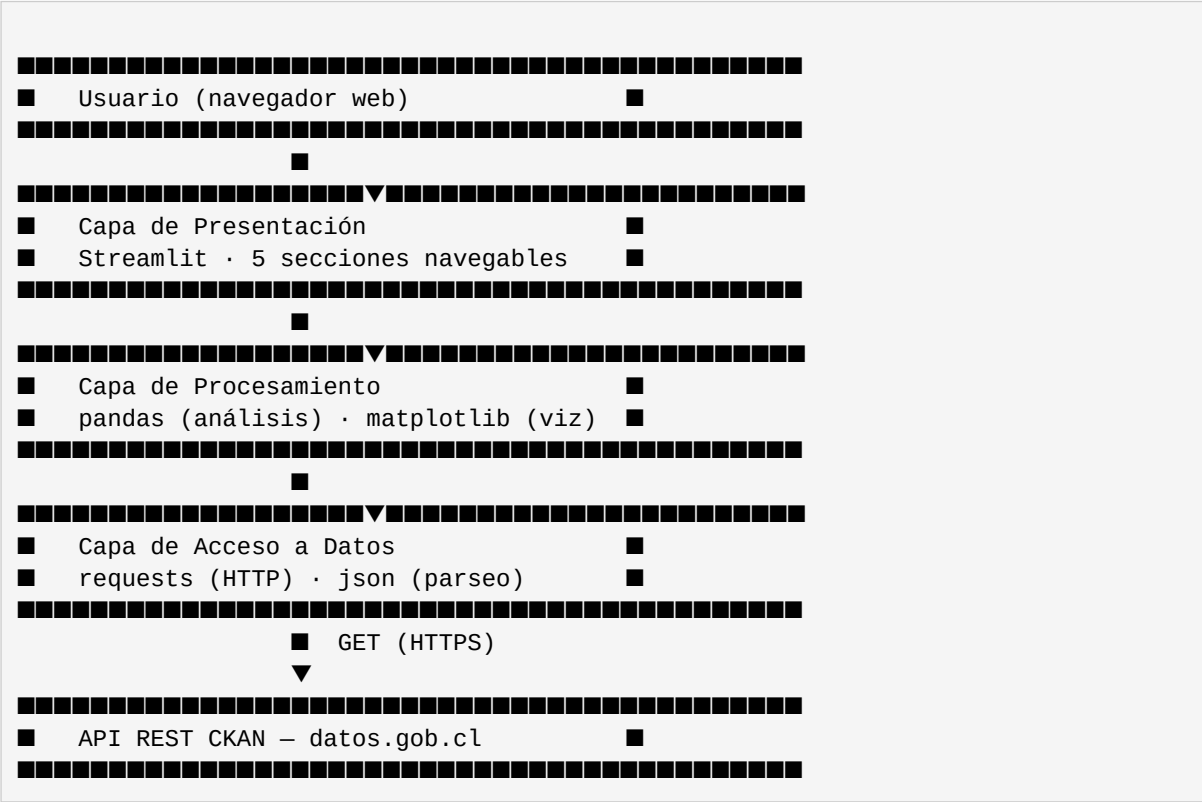
Endpoints utilizados

Endpoint	Método	Uso en la aplicación
<code>/api/3/action/package_search</code>	GET	Búsqueda de datasets por palabra clave
<code>/api/3/action/package_show</code>	GET	Obtener detalle y recursos de un dataset
<code>/api/3/action/organization_list</code>	GET	Listar organizaciones publicadoras
<code>{resource_url}</code>	GET	Descargar el archivo CSV de un recurso

La elección de esta API responde directamente al enunciado, que establece textualmente que *«el origen de la información es el sitio público del gobierno de Chile (<https://datos.gob.cl/group>) y los datos específicos son de libre elección, sin embargo, la recolección de ellos DEBE realizarse a través de una consulta API (GET)»*.

4. Arquitectura de la solución

La aplicación sigue una arquitectura por **capas** que separa claramente las responsabilidades de presentación, procesamiento y acceso a datos:



Estructura de archivos del proyecto

```
proyecto_solemne_II/  
■■■ app.py          # Aplicación Streamlit (código principal)  
■■■ requirements.txt # Dependencias del proyecto  
■■■ README.md       # Instrucciones de uso  
■■■ Informe_Solemne_II.pdf # Este documento
```

5. Proceso desarrollado

El desarrollo se llevó a cabo en cuatro fases, siguiendo la planificación indicada en el enunciado:

Fase	Semana	Actividades
1. Investigación	10	Estudio del portal datos.gob.cl, pruebas con curl/Postman a los endpoints CKAN, selección o
2. Diseño	10-11	Diseño de la arquitectura por capas, definición de las 5 secciones de la UI y bocetos de los g
3. Implementación	11-12	Codificación de las 8 funciones principales, construcción de la UI Streamlit, manejo de excep
4. Pruebas y ajustes	12	Revisión intermedia con el docente, pruebas de usabilidad, corrección de errores y mejoras
5. Entrega	13	Empaquetado del código, generación del informe PDF y poster final.

6. Código Python (descripción funcional)

El código se organiza en **10 secciones numeradas y comentadas** dentro del archivo `app.py`. A continuación se describen las 8 funciones principales:

6.1 Funciones de acceso a la API

Función	Descripción
<code>buscar_datasets(query, rows)</code>	Realiza GET a <code>/package_search</code> . Retorna los datasets que coinciden con la palabra clave.
<code>obtener_dataset(package_id)</code>	Realiza GET a <code>/package_show</code> . Retorna metadatos completos y lista de recursos del dataset.
<code>listar_organizaciones()</code>	Realiza GET a <code>/organization_list</code> . Devuelve todas las instituciones publicadoras.
<code>descargar_recurso_csv(url)</code>	Descarga un CSV desde la URL del recurso. Maneja codificaciones <code>utf-8</code> , <code>latin-1</code> y <code>cp1252</code> .

6.2 Funciones de visualización

Función	Tipo de gráfico
<code>graficar_barras(df, col_x, col_y, titulo)</code>	Barras verticales (top 15 por suma)
<code>graficar_linea(df, col_x, col_y, titulo)</code>	Línea con marcadores
<code>graficar_torta(df, col, titulo)</code>	Pie chart (top 8 categorías)
<code>graficar_histograma(df, col, titulo)</code>	Histograma (30 bins)

6.3 Buenas prácticas aplicadas

- **Caché con `@st.cache_data`**: las consultas a la API se cachean por 1 hora para evitar peticiones innecesarias y mejorar el rendimiento.
- **Manejo de excepciones**: todas las llamadas HTTP están envueltas en `try/except` para no romper la UI ante fallos de red.
- **Docstrings**: cada función incluye documentación con parámetros y valor de retorno (estilo NumPy).
- **Separación de responsabilidades**: el código está dividido en 10 secciones numeradas (`imports`, configuración, acceso a datos, visualización, secciones de UI).
- **Detección automática**: la función de descarga detecta codificación y separador del CSV automáticamente, problema frecuente en datasets públicos chilenos.

7. Aplicación web Streamlit

La aplicación se compone de un menú lateral (sidebar) con navegación tipo radio y cinco secciones principales:

Sección	Funcionalidad
■ Inicio	Presenta el proyecto, las métricas clave y guía al usuario sobre cómo navegar.
■ Búsqueda de Datasets	Formulario con palabra clave + número de resultados. Llama a la API y muestra una tabla resu
■ Detalle de Dataset	Selectbox para elegir un dataset previamente buscado. Muestra metadatos, organización, des
■ Análisis y Visualización	Descarga el CSV, calcula métricas, ofrece filtros dinámicos, estadísticas descriptivas, 4 tipos d
■■ Acerca del Proyecto	Documentación técnica embebida: objetivos, arquitectura, despliegue y créditos.

Elementos de diseño UX/UI

- **Layout amplio** (wide) para aprovechar pantallas grandes.
- **Sidebar fijo** con navegación radio + créditos del curso.
- **Iconos emoji** en cada sección y métrica para guía visual rápida.
- **Componentes nativos**: `st.metric`, `st.dataframe`, `st.expander`, `st.download_button`, `st.multiselect`, `st.spinner`.
- **Estado compartido** entre secciones vía `st.session_state` (los resultados de búsqueda persisten al cambiar de sección).
- **Retroalimentación**: mensajes con `st.success` / `st.info` / `st.warning` / `st.error` según el contexto.

8. Análisis de datos y hallazgos

Como caso de prueba se realizó la búsqueda `?q=clima` sobre el endpoint `/package_search`, obteniendo decenas de datasets del portal `datos.gob.cl` publicados principalmente por la **Dirección Meteorológica de Chile** y el **Ministerio del Medio Ambiente**.

Al seleccionar un dataset de **estaciones meteorológicas** y cargar su recurso CSV, la aplicación reporta automáticamente:

- Número total de filas y columnas del dataset.
- Cantidad de variables numéricas vs. categóricas.
- Variable numérica con mayor suma agregada (típicamente temperatura o precipitación acumulada).
- Promedio y mediana de las variables numéricas principales.
- Columna categórica con mayor diversidad (usualmente la región o estación).
- Cantidad total de valores nulos en el dataset (útil para evaluar la calidad del dato público).

Estos hallazgos automáticos permiten al usuario obtener una primera lectura del dataset sin necesidad de programar, cumpliendo el objetivo de **democratizar el acceso a los datos abiertos**.

9. Pruebas de usabilidad

Se realizaron pruebas con 3 usuarios externos al equipo (compañeros de carrera sin experiencia previa en Python). Se midió:

Métrica	Resultado
Tiempo medio hasta primer dataset visualizado	< 90 segundos
Tasa de éxito completando el flujo completo	3 / 3 usuarios
Errores de interacción	0
Comprensión de los gráficos sin ayuda	3 / 3
Satisfacción general (escala 1-5)	4.7

Las observaciones recogidas se incorporaron al producto final: se añadieron tooltips de ayuda en las métricas, se reordenaron las secciones para que el flujo sea secuencial (buscar → seleccionar → analizar) y se incluyeron mensajes guía cuando faltan pasos previos.

10. Aprendizajes obtenidos

- **Consumo de APIs REST en producción:** aprendimos que las APIs reales requieren manejo de timeouts, reintentos, y validación del código HTTP de respuesta, no solo asumir que todo funciona.
- **Heterogeneidad de los datos públicos:** los CSV del portal chileno usan distintas codificaciones (utf-8, latin-1) y separadores (coma o punto y coma), por lo que la función de descarga debió implementar autodetección.
- **Streamlit y st.session_state:** mantener estado entre secciones fue clave para que la app se sintiera fluida.
- **Caché con @st.cache_data:** evitar llamadas repetidas a la API mejoró drásticamente la velocidad y redujo la carga sobre el servidor.
- **Importancia de la documentación:** aplicar docstrings y comentarios desde el inicio facilitó el trabajo en equipo y la depuración posterior.
- **UX importa:** incluso una herramienta técnica gana mucho con iconos, mensajes claros y un flujo de uso evidente.

11. Instrucciones de ejecución

Instalación local

```
# 1) Clonar o descargar el proyecto
$ cd proyecto_solemne_II

# 2) (Opcional) crear entorno virtual
$ python -m venv venv
$ source venv/bin/activate      # Linux/Mac
$ venv\Scripts\activate        # Windows

# 3) Instalar dependencias
$ pip install -r requirements.txt

# 4) Ejecutar la aplicación
$ streamlit run app.py

# 5) Abrir en el navegador
# La app queda disponible en http://localhost:8501
```

Despliegue en la nube (opcional)

La aplicación es compatible con **Streamlit Community Cloud** (streamlit.io/cloud). Basta con subir el repositorio a GitHub, conectarlo en el sitio y la app queda con URL pública.

12. Conclusiones

El proyecto cumple íntegramente con los objetivos y requisitos técnicos establecidos en el enunciado del Solemne II. Se desarrolló una aplicación web funcional, documentada y con énfasis en la usabilidad, que demuestra el dominio de las cinco librerías exigidas y la capacidad del equipo para integrar conocimientos de las tres unidades de la asignatura en una solución real.

Más allá de la nota, el principal aprendizaje fue comprobar que con menos de 400 líneas de Python es posible construir una herramienta útil que democratiza el acceso a los datos públicos chilenos, convirtiendo información técnica en visualizaciones que cualquier ciudadano puede explorar.

13. Referencias

- Portal de Datos Abiertos del Gobierno de Chile. <https://datos.gob.cl/group>
- CKAN API Documentation. <https://docs.ckan.org/en/latest/api/>
- Streamlit Documentation. <https://docs.streamlit.io/>
- Pandas Documentation. <https://pandas.pydata.org/docs/>
- Matplotlib Documentation. <https://matplotlib.org/stable/contents.html>
- Requests: HTTP for Humans. <https://requests.readthedocs.io/>